



“AÑO DE LA RECUPERACIÓN Y CONSOLIDACIÓN DE LA ECONOMÍA PERUANA”

TÍTULO

SISTEMA DE GESTIÓN DE RECLAMOS DE AGUA POTABLE

DOCENTE:

Rosario Delia Osorio Contreras

INTEGRANTES

García Betancourt Israel Jedidias

Leon Armas Luis Aram

Ramírez Quillatupa Juan Diego

Veliz Durand Vieri Del Piero

CURSO

ANÁLISIS Y DISEÑO DE SOFTWARE

NRC

62151

LINKS

Github: <https://github.com/IsraelGarciaBetancourt/ODS6-Grupo01>

HUANCAYO

2025

ÍNDICE

1. Capitulo 6. Diseño de arquitectura y patrones	1
2. Capitulo 7. Diseño detallado de la base de datos.....	9
3. Capitulo 8. Diseño detallado de sistemas en red y móviles	27
4. Referencias.....	38

SISTEMA DE GESTIÓN DE RECLAMOS DE AGUA POTABLE

1. Capítulo 6. Diseño de arquitectura y patrones

1.1. Estrategia de diseño de software

1.1.1. Estrategia seleccionada

Para el desarrollo del Sistema de Gestión de Reclamos de Agua Potable de SEDAM, se adopta la estrategia de diseño orientada a objetos. Esta estrategia permite modelar el sistema a partir de entidades del mundo real, representadas como clases con atributos, métodos y relaciones claramente definidas.

1.1.2. Justificación de la estrategia

La estrategia orientada a objetos es la más adecuada para este proyecto por las siguientes razones:

a) El sistema trabaja con entidades reales del proceso de SEDAM

El dominio del problema contiene objetos naturales como:

Ciudadano, Operador SEDAM, Reclamo, Notificación, Evidencia, Reporte, etc. Cada uno posee atributos propios y métodos asociados a su comportamiento. La POO facilita la representación de estas entidades de forma clara y fiel.

b) Permite la especialización / herencia

La estrategia OO soporta generalización/especialización, por lo que se implementó:

- USUARIO (clase padre)
- CIUDADANO (hijo)
- OPERADOR_SEDAM (hijo)

Esto permite representar adecuadamente a los dos actores principales del sistema, cada uno con atributos y comportamientos propios. Esto cumple directamente

profundizar el ER y diferenciar claramente los tipos de usuario.

c) Facilita la extensibilidad del sistema

Las clases pueden evolucionar mediante:

- Herencia (extender funcionalidad)
- Composición (objetos que contienen otros objetos)
- Asociaciones (relaciones directas entre entidades)

Esto permite que el sistema crezca sin romper lo ya construido.

d) Soporta adecuadamente procesos con múltiples relaciones

El sistema requiere manejar relaciones como:

- Un ciudadano registra muchos reclamos
- Un reclamo tiene muchos cambios de estado
- Un operador gestiona múltiples reclamos
- Un reclamo puede generar múltiples notificaciones
- Un reporte está ligado a un formato oficial

La POO modela estas relaciones mediante asociación, agregación o composición según corresponda.

e) Adecuación de la estrategia orientada a objetos al tipo de sistema

La estrategia orientada a objetos se ajusta adecuadamente al sistema de gestión de reclamos debido a que permite representar de manera natural las entidades involucradas como ciudadanos, operadores, reclamos, estados y notificaciones mediante clases que encapsulan atributos y comportamientos. Esto facilita la organización del software en componentes claros y coherentes.

Asimismo, el sistema requiere evolucionar con el tiempo, incorporando nuevos tipos de reclamos, estados del proceso o formatos oficiales. La orientación a objetos proporciona los mecanismos necesarios para esta extensibilidad, como la herencia, la composición y la

asociación, permitiendo ampliar funcionalidades sin afectar la estructura existente.

Finalmente, la orientación a objetos favorece la reutilización de componentes por ejemplo, mediante la clase base USUARIO y sus especializaciones, optimizando el desarrollo y manteniendo la consistencia del sistema.

f) Contribución de la estrategia orientada a objetos al cumplimiento de los principios de calidad del diseño

La orientación a objetos permite cumplir de manera efectiva los principios de calidad del diseño. Favorece la modularidad, ya que el sistema se organiza en clases y componentes independientes; mejora la cohesión, asignando responsabilidades claras a cada clase; y reduce el acoplamiento, estableciendo relaciones controladas entre los módulos.

Además, aplica abstracción y encapsulamiento, protegiendo los datos y ocultando detalles internos de implementación. Finalmente, facilita la reutilización del código mediante herencia y composición, permitiendo extender funcionalidades sin reconstruir el sistema.

1.2. Tipo de arquitectura del sistema

1.2.1. Arquitectura seleccionada

Para el desarrollo del Sistema de Gestión de Reclamos de Agua Potable para la EPS SEDAM se adopta la Arquitectura en Capas (N-Tier). Este patrón organiza el sistema en niveles con responsabilidades claramente definidas, permitiendo una mayor mantenibilidad, escalabilidad y separación lógica entre la presentación, la lógica de negocio y el acceso a datos.

1.2.2. Justificación de la elección de arquitectura

a. Escalabilidad

La separación por capas permite distribuir la carga en distintos servidores según la necesidad, facilitando el crecimiento gradual del sistema ante un aumento de reclamos ciudadanos.

b. Mantenibilidad

Cada capa puede modificarse sin afectar a las demás. Esto facilita:

- corregir errores
- añadir nuevas funcionalidades
- actualizar las reglas de negocio institucionales
- adaptarse a cambios normativos de SEDAM

c. Seguridad

Aislar presentación, lógica y datos permite aplicar controles como:

- autenticación y autorización
- protección de datos sensibles
- filtros contra inyecciones
- validaciones internas estructuradas

d. Cohesión y bajo acoplamiento

El diseño por capas mantiene un código más organizado y coherente con la estrategia orientada a objetos que estructura las entidades del dominio y sus interacciones.

e. Adecuación al contexto institucional

SEDAM es una única EPS con un sistema centralizado. Por ello, una arquitectura más compleja (microservicios, hexagonal o event-driven) sería innecesaria y costosa. La arquitectura en capas proporciona el balance adecuado entre simplicidad, orden, robustez y claridad técnica, alineándose con los objetivos del sistema y del ODS 6.

La Arquitectura en Capas (N-Tier) ofrece una estructura sólida y escalable para el Sistema de Gestión de Reclamos de Agua Potable de la EPS SEDAM. Se adapta adecuadamente al contexto institucional, garantiza mantenibilidad, soporte a largo plazo y seguridad, y es plenamente compatible con el diseño orientado a objetos adoptado en el proyecto.

1.2.3. Descripción de la Arquitectura

I. Capa de Presentación

Responsable de la interacción con los usuarios, tanto ciudadanos como operadores SEDAM. Incluye:

- Interfaces web responsivas.
- Formularios de registro de reclamos.
- Paneles de consulta y seguimiento.
- Pantallas de gestión para operadores.

Esta capa puede implementar el patrón MVC para separar vista, controlador y modelo, evitando mezclar lógica visual con reglas de negocio.

II. Capa de Lógica de Negocio

Contiene las reglas que rigen el funcionamiento interno del sistema:

- Validación y registro de reclamos.
- Generación del código de seguimiento.
- Gestión de estados y flujos internos.
- Registro del historial y trazabilidad.
- Emisión de notificaciones.
- Gestión de formatos oficiales de SEDAM.
- Generación de reportes institucionales.

Es la capa más importante, ya que implementa los procesos definidos en el diseño orientado a objetos (clases, relaciones y responsabilidades del dominio).

III. Capa de Acceso a Datos

Encargada de la comunicación con la base de datos relacional. Incluye:

- Repositorios / DAOs.
- Sentencias SQL optimizadas.
- Manejo de transacciones.
- Consultas y persistencia de entidades.

Esta capa abstrae el acceso físico a la información, protegiendo la lógica de negocio de modificaciones en la infraestructura de datos.

IV. Base de Datos

Modelo relacional normalizado (3FN) que almacena:

- Usuarios (ciudadanos y operadores)
- Reclamos
- Estados e historial
- Evidencias
- Notificaciones
- Reportes institucionales
- Formatos oficiales de SEDAM

Su diseño responde a la profundización realizada previamente.

1.2.4. Diagrama General de la Arquitectura

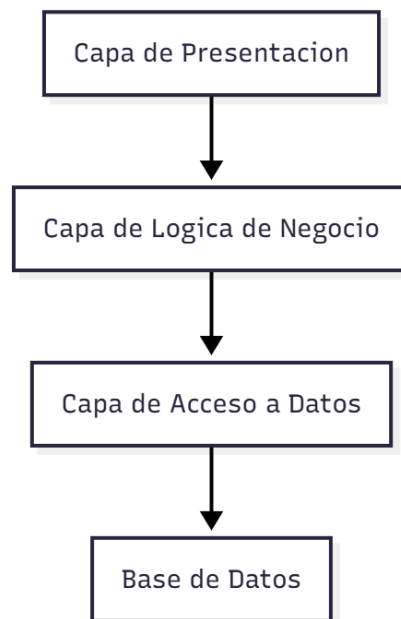


Figura 1. En el diagrama se observa de manera general como estaría armado la arquitectura por n-capas

1.3. Patrones de diseño aplicados

En el desarrollo del Sistema de Gestión de Reclamos para SEDAM, se aplicaron diversos patrones de diseño con el objetivo de mejorar la mantenibilidad, escalabilidad y claridad del sistema. A continuación, se

describen los patrones seleccionados, su propósito y el aporte que brindan a la solución.

a. Patrón Observer (Observador)

Propósito

Permite que un objeto (Sujeto) notifique automáticamente a otros objetos interesados (Observadores) cuando ocurre un cambio de estado.

Aplicación en el sistema

En el módulo de reclamos, cada vez que el estado de un reclamo cambia (por ejemplo: *Registrado* → *En evaluación* → *Atendido*), el sistema debe enviar notificaciones automáticas al ciudadano o al operador correspondiente. El reclamo actúa como Sujeto, y los módulos de notificaciones como Observadores.

Aporte al sistema

- Garantiza una comunicación automática entre módulos.
- Reduce el acoplamiento entre “Reclamo” y “Notificación”.
- Facilita agregar nuevos medios de notificación (correo, web, SMS) sin modificar el código central.

b. Patrón DAO (Data Access Object)

Propósito

Encapsular la lógica de acceso a datos en clases especializadas que permiten separar la lógica de persistencia de la lógica de negocio.

Aplicación en el sistema

Las entidades principales del sistema (Usuario, Reclamo, Evidencia, Notificación, Historial, etc.) interactúan con la base de datos mediante objetos DAO. Por ejemplo, el objeto ReclamoDAO se encarga de registrar, actualizar, obtener y cerrar reclamos sin exponer detalles del motor de base de datos.

Aporte al sistema

- Facilita el mantenimiento y la reutilización del código.
- Permite cambiar o mejorar la base de datos sin afectar la lógica de negocio.
- Reduce el acoplamiento entre capas y fortalece la arquitectura en capas (N-Tier).

c. Patrón Fachada (Facade)

Propósito

Proporcionar una interfaz única y simplificada para un conjunto complejo de operaciones internas.

Aplicación en el sistema

El proceso de registrar un reclamo implica varios pasos:

1. Registrar los datos del reclamo.
2. Guardar evidencias.
3. Crear el historial inicial.
4. Notificar al ciudadano.

En lugar de que la interfaz gestione cada paso, se implementa una Fachada de Gestión de Reclamos que centraliza todas estas operaciones en un único punto de acceso.

Aporte al sistema

- Simplifica la interacción con procesos complejos.
- Reduce errores al estandarizar el flujo interno.
- Mejora la mantenibilidad al concentrar la lógica de coordinación en una sola clase.

1.4. Diseño estructural

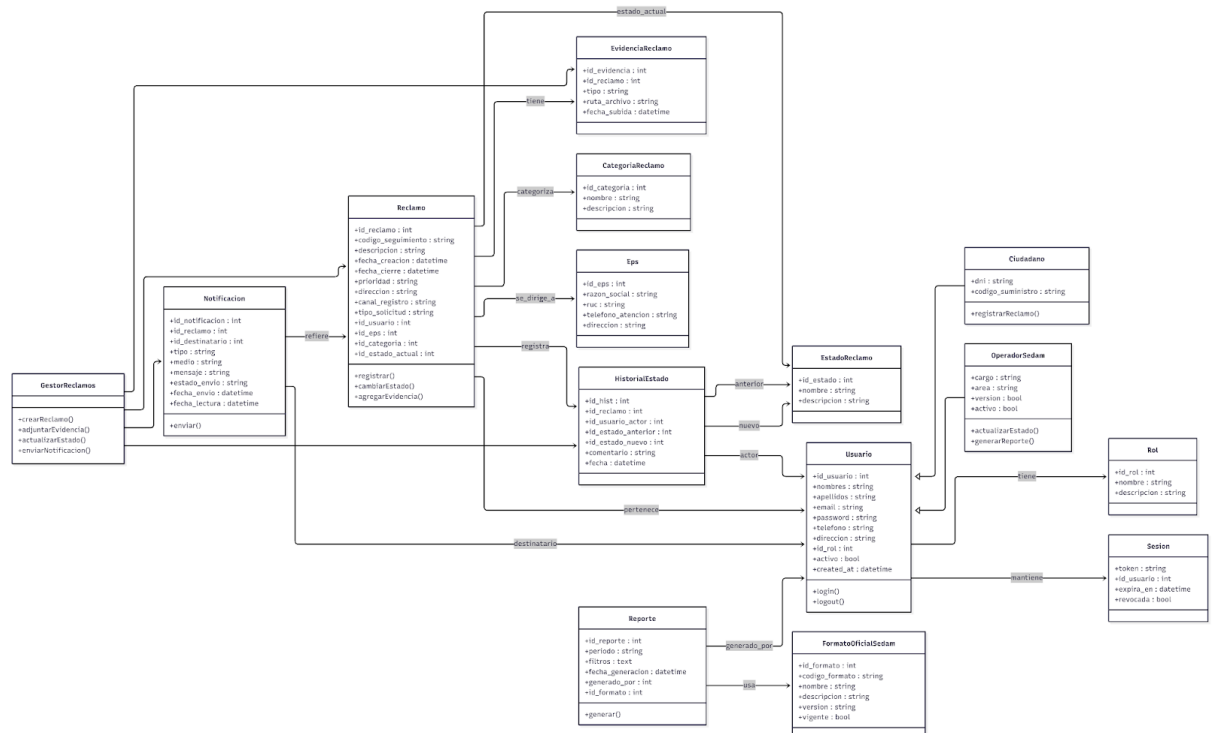


Figura 2. La imagen muestra el diseño estructural del proyecto

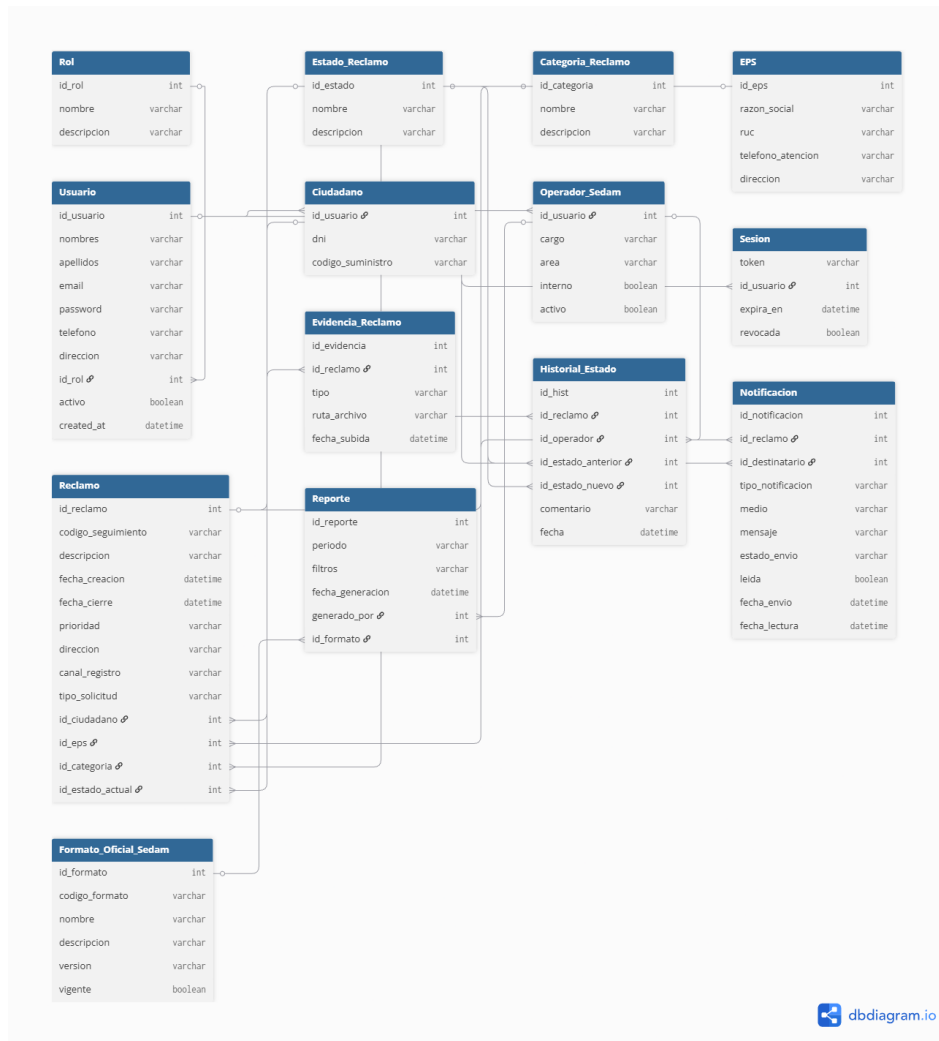
<https://www.mermaidchart.com/app/projects/989bc838-7a69-462b-aec0-859a513dd7c0/diagrams/12adfcdf-ea14-49e8-be74-327438341677/share/invite/eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJkb2N1bWVudEIEljoMTJhZGZjYWYtZWExNC00OWU4LWJINzQtMzI3NDM4MzQxNjc3IiwiaWF0Ij0iZm9udC9weKGVsqBBsUUjwEB5HYat1t9hvEQclo>

2. Capítulo 7. Diseño detallado de la base de datos

1. Modelo lógico y físico

1.1. Modelo lógico

El modelo entidad–relación del sistema representa de forma conceptual las principales entidades involucradas en la gestión de reclamos, como usuarios, ciudadanos, operadores, reclamos, estados, categorías, evidencias, notificaciones y reportes. El diagrama muestra cómo estas entidades se relacionan entre sí: un ciudadano registra reclamos, un operador actualiza su historial, un reclamo genera evidencias y notificaciones, y los reportes se elaboran a partir de formatos oficiales. Este modelo ofrece una visión clara de la estructura lógica del sistema y sirve como base para el diseño físico de la base de datos.



1.2. Modelo Físico

El modelo físico traduce el modelo lógico a estructuras concretas del motor MySQL, definiendo tipos de datos, claves primarias, claves foráneas, índices y restricciones. Se utilizará el motor InnoDB para soportar transacciones ACID e integridad referencial, y el conjunto de caracteres utf8mb4 para soportar caracteres especiales.

- **Motor de almacenamiento:** InnoDB (soporta FK, transacciones y bloqueos a nivel de fila).
- **Codificación:** utf8mb4 y utf8mb4_unicode_ci para compatibilidad con caracteres en español.
- **Tipo de claves primarias:** INT con AUTO_INCREMENT para la mayoría de las tablas.
- **Campos booleanos:** representados como TINYINT(1) en MySQL.

- **Fechas y horas:** DATETIME para registrar fecha y hora completas.
- **Textos largos:** TEXT para descripciones y mensajes.
- **Índices:** claves foráneas generan índices implícitos; se pueden agregar índices adicionales sobre campos de búsqueda frecuente (ej. codigo_seguimiento, email, dni).

El diagrama físico muestra la estructura final de la base de datos del sistema de gestión de reclamos, representada mediante tablas relacionales con sus respectivas claves primarias, claves foráneas y tipos de datos. En él se visualizan las relaciones entre usuarios, ciudadanos, operadores, reclamos, evidencias, notificaciones y reportes, asegurando la integridad y coherencia de la información. Este modelo refleja cómo se implementará la base de datos en MySQL y sirve como guía para la creación del esquema mediante instrucciones SQL.

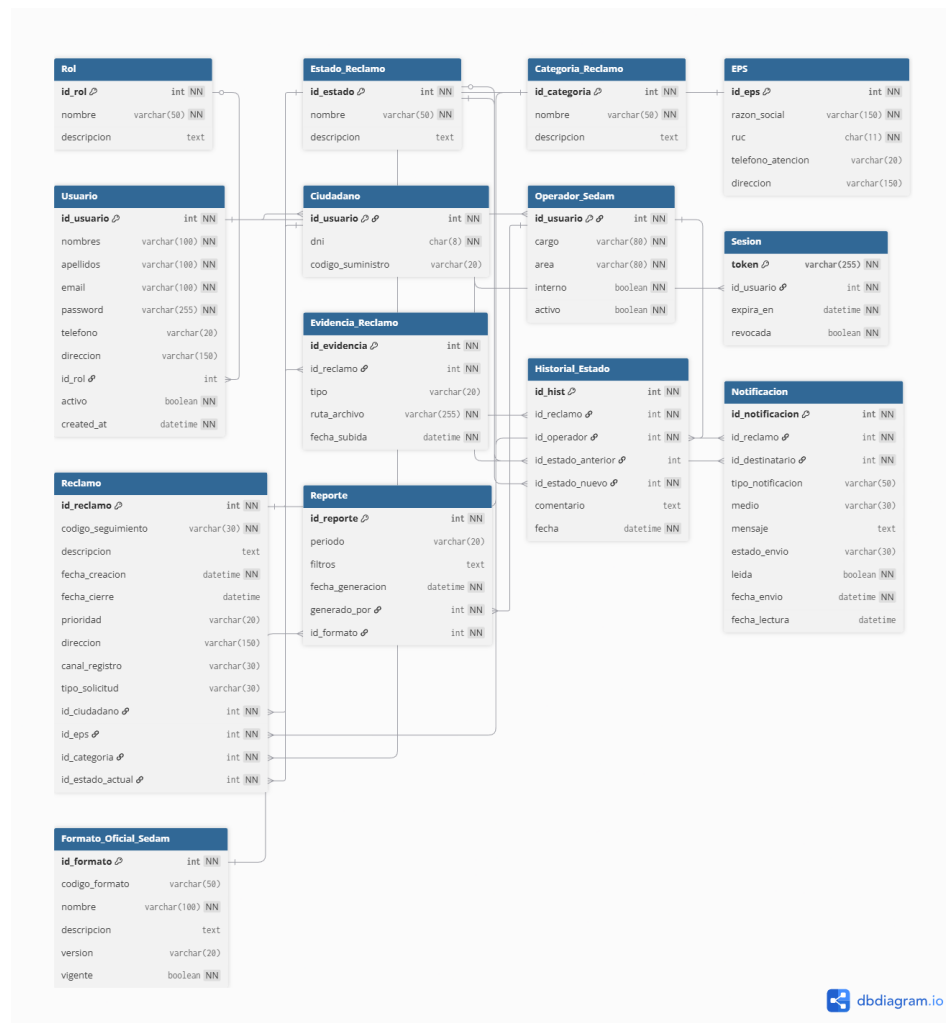


Diagrama Relacional

<https://github.com/IsraelGarciaBetancourt/ODS6-Grupo01/blob/main/Modelo%20F%C3%ADsico%20Diagrama%20Relacional.png>

2. Script SQL

2.1. Crear base de datos

```
CREATE DATABASE IF NOT EXISTS sedam_reclamos
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_unicode_ci;

USE sedam_reclamos;
```

2.2. Tablas maestras

2.2.1. Tabla ROL

```
CREATE TABLE rol (
  id_rol INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  descripcion TEXT
) ENGINE=InnoDB;
```

2.2.2. Tabla Estado Reclamo

```
CREATE TABLE estado_reclamo (
  id_estado INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(50) NOT NULL,
  descripcion TEXT
) ENGINE=InnoDB;
```

2.2.3. Tabla CATEGORIA_RECLAMO

```
CREATE TABLE categoria_reclamo (  
    id_categoria INT AUTO_INCREMENT PRIMARY KEY,  
    nombre VARCHAR(50) NOT NULL,  
    descripcion TEXT  
) ENGINE=InnoDB;
```

2.2.4. Tabla EPS

```
CREATE TABLE eps (  
    id_eps INT AUTO_INCREMENT PRIMARY KEY,  
    razon_social VARCHAR(150) NOT NULL,  
    ruc CHAR(11) NOT NULL,  
    telefono_atencion VARCHAR(20),  
    direccion VARCHAR(150)  
) ENGINE=InnoDB;
```

2.3. Tablas de usuarios y autenticación

2.3.1. Tabla Usuario

```
CREATE TABLE usuario (  
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,  
    nombres VARCHAR(100) NOT NULL,  
    apellidos VARCHAR(100) NOT NULL,  
    email VARCHAR(100) NOT NULL UNIQUE,  
    password VARCHAR(255) NOT NULL,  
    telefono VARCHAR(20),  
    direccion VARCHAR(150),  
    id_rol INT NULL,  
    activo TINYINT(1) NOT NULL DEFAULT 1,  
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_usuario_rol FOREIGN KEY (id_rol)  
        REFERENCES rol(id_rol)  
        ON UPDATE CASCADE ON DELETE SET NULL  
) ENGINE=InnoDB;
```

2.3.2. Tabla Ciudadano

```
CREATE TABLE ciudadano (  
    id_usuario INT PRIMARY KEY,  
    dni CHAR(8) NOT NULL UNIQUE,  
    codigo_suministro VARCHAR(20),  
    CONSTRAINT fk_ciudadano_usuario FOREIGN KEY (id_usuario)  
        REFERENCES usuario(id_usuario)  
        ON UPDATE CASCADE ON DELETE CASCADE  
) ENGINE=InnoDB;
```

2.3.3. Tabla OPERADOR_SEDAM

```
CREATE TABLE operador_sedam (  
    id_usuario INT PRIMARY KEY,  
    cargo VARCHAR(80) NOT NULL,  
    area VARCHAR(80) NOT NULL,  
    interno TINYINT(1) NOT NULL DEFAULT 1,  
    activo TINYINT(1) NOT NULL DEFAULT 1,  
    CONSTRAINT fk_operador_usuario FOREIGN KEY (id_usuario)  
        REFERENCES usuario(id_usuario)  
        ON UPDATE CASCADE ON DELETE CASCADE  
) ENGINE=InnoDB;
```

2.3.4. Tabla SESION

```
CREATE TABLE sesion (  
    token VARCHAR(255) PRIMARY KEY,  
    id_usuario INT NOT NULL,  
    expira_en DATETIME NOT NULL,  
    revocada TINYINT(1) NOT NULL DEFAULT 0,  
    CONSTRAINT fk_sesion_usuario FOREIGN KEY (id_usuario)  
        REFERENCES usuario(id_usuario)  
        ON UPDATE CASCADE ON DELETE CASCADE  
) ENGINE=InnoDB;
```

2.4. Tablas principales del reclamo

2.4.1. Tabla RECLAMO

```
CREATE TABLE reclamo (  
    id_reclamo INT AUTO_INCREMENT PRIMARY KEY,  
    codigo_seguimiento VARCHAR(30) NOT NULL UNIQUE,  
    descripcion TEXT,  
    fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    fecha_cierre DATETIME NULL,  
    prioridad VARCHAR(20),  
    direccion VARCHAR(150),  
    canal_registro VARCHAR(30),  
    tipo_solicitud VARCHAR(30),  
    id_ciudadano INT NOT NULL,  
    id_eps INT NOT NULL,  
    id_categoria INT NOT NULL,  
    id_estado_actual INT NOT NULL,  
    CONSTRAINT fk_reclamo_ciudadano FOREIGN KEY (id_ciudadano)  
        REFERENCES ciudadano(id_usuario)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
    CONSTRAINT fk_reclamo_eps FOREIGN KEY (id_eps)  
        REFERENCES eps(id_eps)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
    CONSTRAINT fk_reclamo_categoria FOREIGN KEY (id_categoria)  
        REFERENCES categoria_reclamo(id_categoria)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
    CONSTRAINT fk_reclamo_estado FOREIGN KEY (id_estado_actual)  
        REFERENCES estado_reclamo(id_estado)  
        ON UPDATE CASCADE ON DELETE RESTRICT  
) ENGINE=InnoDB;
```

2.4.2. Tabla EVIDENCIA_RECLAMO

```
CREATE TABLE evidencia_reclamo (  
    id_evidencia INT AUTO_INCREMENT PRIMARY KEY,  
    id_reclamo INT NOT NULL,  
    tipo VARCHAR(20),  
    ruta_archivo VARCHAR(255) NOT NULL,  
    fecha_subida DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    CONSTRAINT fk_evidencia_reclamo FOREIGN KEY (id_reclamo)  
        REFERENCES reclamo(id_reclamo)  
        ON UPDATE CASCADE ON DELETE CASCADE  
) ENGINE=InnoDB;
```

2.4.3. Tabla HISTORIAL_ESTADO

```
CREATE TABLE historial_estado (  
    id_hist INT AUTO_INCREMENT PRIMARY KEY,  
    id_reclamo INT NOT NULL,  
    id_operador INT NOT NULL,  
    id_estado_anterior INT NULL,  
    id_estado_nuevo INT NOT NULL,  
    comentario TEXT,  
    fecha DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
    CONSTRAINT fk_hist_reclamo FOREIGN KEY (id_reclamo)  
        REFERENCES reclamo(id_reclamo)  
        ON UPDATE CASCADE ON DELETE CASCADE,  
  
    CONSTRAINT fk_hist_operador FOREIGN KEY (id_operador)  
        REFERENCES operador_sedam(id_usuario)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
  
    CONSTRAINT fk_hist_estado_anterior FOREIGN KEY (id_estado_anterior)  
        REFERENCES estado_reclamo(id_estado)  
        ON UPDATE CASCADE ON DELETE SET NULL,  
  
    CONSTRAINT fk_hist_estado_nuevo FOREIGN KEY (id_estado_nuevo)  
        REFERENCES estado_reclamo(id_estado)  
        ON UPDATE CASCADE ON DELETE RESTRICT  
) ENGINE=InnoDB;
```

2.4.4. Tabla NOTIFICACION

```
CREATE TABLE notificacion (  
    id_notificacion INT AUTO_INCREMENT PRIMARY KEY,  
    id_reclamo INT NOT NULL,  
    id_destinatario INT NOT NULL,  
    tipo_notificacion VARCHAR(50),  
    medio VARCHAR(30),  
    mensaje TEXT,  
    estado_envio VARCHAR(30),  
    leida TINYINT(1) NOT NULL DEFAULT 0,  
    fecha_envio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    fecha_lectura DATETIME NULL,  
  
    CONSTRAINT fk_notif_reclamo FOREIGN KEY (id_reclamo)  
        REFERENCES reclamo(id_reclamo)  
        ON UPDATE CASCADE ON DELETE CASCADE,  
  
    CONSTRAINT fk_notif_usuario FOREIGN KEY (id_destinatario)  
        REFERENCES usuario(id_usuario)  
        ON UPDATE CASCADE ON DELETE CASCADE  
) ENGINE=InnoDB;
```

2.5. Tablas de reportes

2.5.1. Tabla FORMATO_OFICIAL_SEDAM

```
CREATE TABLE formato_oficial_sedam (  
    id_formato INT AUTO_INCREMENT PRIMARY KEY,  
    codigo_formato VARCHAR(50),  
    nombre VARCHAR(100) NOT NULL,  
    descripcion TEXT,  
    version VARCHAR(20),  
    vigente TINYINT(1) NOT NULL DEFAULT 1  
) ENGINE=InnoDB;
```

2.5.2. Tabla REPORTE

```
CREATE TABLE reporte (  
    id_reporte INT AUTO_INCREMENT PRIMARY KEY,  
    periodo VARCHAR(20),  
    filtros TEXT,  
    fecha_generacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    generado_por INT NOT NULL,  
    id_formato INT NOT NULL,  
  
    CONSTRAINT fk_rep_operador FOREIGN KEY (generado_por)  
        REFERENCES operador_sedam(id_usuario)  
        ON UPDATE CASCADE ON DELETE RESTRICT,  
  
    CONSTRAINT fk_rep_formato FOREIGN KEY (id_formato)  
        REFERENCES formato_oficial_sedam(id_formato)  
        ON UPDATE CASCADE ON DELETE RESTRICT  
) ENGINE=InnoDB;
```

2.6. Datos iniciales (INSERTS)

2.6.1. Roles

```
INSERT INTO rol (nombre, descripcion) VALUES  
('Administrador', 'Acceso total al sistema'),  
('Operador', 'Gestiona reclamos'),  
('Ciudadano', 'Usuario común que registra reclamos');
```

2.6.2. Estados iniciales del reclamo

```
INSERT INTO estado_reclamo (nombre, descripcion) VALUES  
('Registrado', 'Reclamo ingresado en el sistema'),  
('En evaluación', 'Reclamo en revisión por operador'),  
('Atendido', 'Reclamo resuelto'),  
('Cerrado', 'Proceso finalizado');
```

2.6.3. Categorías de reclamo

```
INSERT INTO categoria_reclamo (nombre, descripcion) VALUES  
( 'Facturación', 'Problemas relacionados con el monto del recibo'),  
( 'Corte de servicio', 'Interrupciones en el servicio de agua'),  
( 'Calidad del agua', 'Color, olor u otros problemas de calidad'),  
( 'Conexiones', 'Problemas con instalaciones o tuberías');
```

2.6.4. Registrar SEDAM como EPS única

```
INSERT INTO eps (razon_social, ruc, telefono_atencion, direccion)  
VALUES ('SEDAM HUANCAYO S.A.', '20123456789', '064-123456', 'Av. Mariscal Castilla 123');
```

3. Procedimientos almacenados, vistas y triggers

3.1. Procedimientos Almacenados

3.1.1. SP: Registrar un reclamo

Genera automáticamente un código de seguimiento, inserta el reclamo y devuelve el ID generado.

```
DELIMITER $$

CREATE PROCEDURE sp_registrar_reclamo(
    IN p_descripcion TEXT,
    IN p_prioridad VARCHAR(20),
    IN p_direccion VARCHAR(150),
    IN p_canal_registro VARCHAR(30),
    IN p_tipo_solicitud VARCHAR(30),
    IN p_id_ciudadano INT,
    IN p_id_eps INT,
    IN p_id_categoria INT
)
BEGIN
    DECLARE v_codigo VARCHAR(30);

    -- Generar código: REC + timestamp
    SET v_codigo = CONCAT('REC', UNIX_TIMESTAMP());

    INSERT INTO reclamo(
        codigo_seguimiento, descripcion, prioridad, direccion,
        canal_registro, tipo_solicitud, id_ciudadano, id_eps,
        id_categoria, id_estado_actual
    ) VALUES (
        v_codigo, p_descripcion, p_prioridad, p_direccion,
        p_canal_registro, p_tipo_solicitud, p_id_ciudadano,
        p_id_eps, p_id_categoria, 1
    );

    SELECT LAST_INSERT_ID() AS id_reclamo, v_codigo AS codigo_seguimiento;
END$$

DELIMITER ;
```



3.1.2. SP: Cambiar el estado de un reclamo

Registra el historial, valida transición y actualiza estado.

```
DELIMITER $$

CREATE PROCEDURE sp_cambiar_estado(
    IN p_id_reclamo INT,
    IN p_id_operador INT,
    IN p_estado_nuevo INT,
    IN p_comentario TEXT
)
BEGIN
    DECLARE v_estado_actual INT;

    SELECT id_estado_actual
    INTO v_estado_actual
    FROM reclamo
    WHERE id_reclamo = p_id_reclamo;

    -- Registrar historial
    INSERT INTO historial_estado(
        id_reclamo, id_operador, id_estado_anterior,
        id_estado_nuevo, comentario
    ) VALUES (
        p_id_reclamo, p_id_operador, v_estado_actual,
        p_estado_nuevo, p_comentario
    );

    -- Actualizar estado actual
    UPDATE reclamo
    SET id_estado_actual = p_estado_nuevo
    WHERE id_reclamo = p_id_reclamo;

END$$

DELIMITER ;
```



3.1.3. SP: Registrar evidencia de reclamo

```
DELIMITER $$

CREATE PROCEDURE sp_registrar_evidencia(
    IN p_id_reclamo INT,
    IN p_tipo VARCHAR(20),
    IN p_ruta VARCHAR(255)
)
BEGIN
    INSERT INTO evidencia_reclamo(id_reclamo, tipo, ruta_archivo)
    VALUES(p_id_reclamo, p_tipo, p_ruta);
END$$

DELIMITER ;
```

3.1.4. SP: Registrar notificación

```
DELIMITER $$

CREATE PROCEDURE sp_registrar_notificacion(
    IN p_id_reclamo INT,
    IN p_id_destinatario INT,
    IN p_tipo VARCHAR(50),
    IN p_medio VARCHAR(30),
    IN p_mensaje TEXT
)
BEGIN
    INSERT INTO notificacion(
        id_reclamo, id_destinatario, tipo_notificacion,
        medio, mensaje, estado_envio
    ) VALUES (
        p_id_reclamo, p_id_destinatario, p_tipo,
        p_medio, p_mensaje, 'Pendiente'
    );
END$$

DELIMITER ;
```

3.2. Vistas (Views)

3.2.1. Vista: Reclamos detallados

Muestra información completa para dashboards del operador.

```
CREATE VIEW vw_reclamos_detallados AS
SELECT
    r.id_reclamo,
    r.codigo_seguimiento,
    r.descripcion,
    r.fecha_creacion,
    r.prioridad,
    r.canal_registro,
    r.tipo_solicitud,

    c.dni AS dni_ciudadano,
    u.nombres AS ciudadano_nombres,
    u.apellidos AS ciudadano_apellidos,

    cat.nombre AS categoria,
    est.nombre AS estado,
    eps.razon_social AS eps
FROM reclamo r
JOIN ciudadano c ON r.id_ciudadano = c.id_usuario
JOIN usuario u ON u.id_usuario = c.id_usuario
JOIN categoria_reclamo cat ON r.id_categoria = cat.id_categoria
JOIN estado_reclamo est ON r.id_estado_actual = est.id_estado
JOIN eps ON eps.id_eps = r.id_eps;
```

3.2.2. Vista: Historial de reclamos

```
CREATE VIEW vw_historial_reclamo AS
SELECT
    h.id_hist,
    h.id_reclamo,
    r.codigo_seguimiento,
    h.id_operador,
    u.nombres AS operador,
    h.id_estado_anterior,
    h.id_estado_nuevo,
    estA.nombre AS estado_anterior,
    estN.nombre AS estado_nuevo,
    h.comentario,
    h.fecha
FROM historial_estado h
JOIN reclamo r ON r.id_reclamo = h.id_reclamo
JOIN usuario u ON u.id_usuario = h.id_operador
LEFT JOIN estado_reclamo estA ON estA.id_estado = h.id_estado_anterior
JOIN estado_reclamo estN ON estN.id_estado = h.id_estado_nuevo;
```

3.2.3. Vista: Notificaciones del usuario

```
CREATE VIEW vw_notificaciones_usuario AS
SELECT
    n.id_notificacion,
    n.id_destinatario,
    u.nombres,
    u.apellidos,
    n.id_reclamo,
    r.codigo_seguimiento,
    n.tipo_notificacion,
    n.medio,
    n.mensaje,
    n.estado_envio,
    n.leida,
    n.fecha_envio,
    n.fecha_lectura
FROM notificacion n
JOIN usuario u ON u.id_usuario = n.id_destinatario
JOIN reclamo r ON r.id_reclamo = n.id_reclamo;
```

3.3. Triggers

3.3.1. Trigger: Registrar historial automáticamente cuando cambia el estado

```
DELIMITER $$

CREATE TRIGGER tr_reclamo_cambio_estado
AFTER UPDATE ON reclamo
FOR EACH ROW
BEGIN
    IF NEW.id_estado_actual <> OLD.id_estado_actual THEN
        INSERT INTO historial_estado(
            id_reclamo, id_operador, id_estado_anterior,
            id_estado_nuevo, comentario
        ) VALUES (
            NEW.id_reclamo,
            NULL,
            OLD.id_estado_actual,
            NEW.id_estado_actual,
            'Cambio automático de estado'
        );
    END IF;
END$$

DELIMITER ;
```

3.3.2. Trigger: Registrar fecha de cierre cuando se marca como “Cerrado”

```
DELIMITER $$

CREATE TRIGGER tr_reclamo_fecha_cierre
BEFORE UPDATE ON reclamo
FOR EACH ROW
BEGIN
    IF NEW.id_estado_actual = 4 AND OLD.fecha_cierre IS NULL THEN
        SET NEW.fecha_cierre = NOW();
    END IF;
END$$

DELIMITER ;
```

3.3.3. Trigger: Timestamp automático en evidencias

```
DELIMITER $$

CREATE TRIGGER tr_evidencia_timestamp
BEFORE INSERT ON evidencia_reclamo
FOR EACH ROW
BEGIN
    SET NEW.fecha_subida = NOW();
END$$

DELIMITER ;
```

3.3.4. Trigger: Marcar notificación como enviada si medio = 'Email'

```
DELIMITER $$

CREATE TRIGGER tr_notificacion_envio
AFTER INSERT ON notificacion
FOR EACH ROW
BEGIN
    IF NEW.medio = 'Email' THEN
        UPDATE notificacion
        SET estado_envio = 'Enviado'
        WHERE id_notificacion = NEW.id_notificacion;
    END IF;
END$$

DELIMITER ;
```

4. Seguridad y respaldo

4.1. Seguridad de la base de datos

La seguridad se gestiona desde tres frentes: protección de acceso, privilegios por roles y seguridad interna del motor MySQL.

4.1.1. Control de acceso

- **Los usuarios del sistema web NO acceden directamente a MySQL.** Solo el servidor backend interactúa con la base de datos.

- Se creó un usuario interno exclusivo para la aplicación:

```
CREATE USER 'sedam_app'@'localhost' IDENTIFIED BY 'ClaveSegura2025!';
```

- Se aplican reglas estrictas de firewall:
 - Solo el servidor backend puede conectarse al puerto 3306.
 - Accesos externos quedan bloqueados.
- Las conexiones de la aplicación usan **SSL/TLS** para evitar sniffing de información (contraseñas, tokens, datos personales).

4.1.2. Privilegios mínimos (Principio de menor privilegio)

Usuario de la aplicación

```
GRANT SELECT, INSERT, UPDATE, DELETE  
ON sedam_reclamos.*  
TO 'sedam_app'@'localhost';
```

No se entrega permiso para:

- ✗ ALTER
- ✗ DROP
- ✗ CREATE
- ✗ GRANT
- ✗ SUPER

Esto evita:

- Alteración no deseada de tablas
- Pérdida accidental de datos
- Escalación de privilegios

4.1.3. Roles internos para auditoría

Rol para administración

```
CREATE ROLE 'rol_admin_db';  
GRANT ALL PRIVILEGES ON sedam_reclamos.* TO 'rol_admin_db';
```

Rol para auditoría de logs

```
CREATE ROLE 'rol_auditor';  
GRANT SELECT ON sedam_reclamos.* TO 'rol_auditor';
```

Un auditor puede revisar tablas como:

- historial_estado
- notificacion
- sesion

pero **no puede modificar datos**.

4.1.4. Cifrado y protección de datos sensibles

a) Hash de contraseñas

Las contraseñas NO se almacenan en texto plano.

La aplicación debe usar:

- bcrypt
o
- argon2

El campo password en la tabla USUARIO soporta hashes largos:

password VARCHAR(255)

b) Datos sensibles protegidos

- DNI
- Código de suministro
- Token de sesión
- Dirección del usuario
- Evidencias de reclamo

Estos datos pueden encriptarse a nivel de aplicación, dependiendo del nivel de seguridad requerido por SEDAM.

4.1.5. Auditoría y trazabilidad

El sistema registra toda acción importante:

- Cambios de estado en un reclamo → tabla historial_estado
- Evidencias agregadas → evidencia_reclamo
- Notificaciones generadas → notificacion
- Sesiones activas → sesion

Esto permite auditorías posteriores y asegura cumplimiento normativo.

4.2. Respaldo (Backup)

4.2.1. Estrategia de respaldos

1) Respaldo completo diario (Full Backup)

Ejecutado durante la madrugada:

```
mysqldump -u root -p sedam_reclamos >  
backup_sedam_full.sql
```

2) Respaldos incrementales cada 4 horas

Usando los binlogs de MySQL:

```
mysqlbinlog binlog.000001 > backup_incremental_1.sql
```

3) Copia semanal fuera del servidor

En un almacenamiento externo:

- Google Drive institucional
- Servidor NAS
- Disco duro externo cifrado

4) Regla 3-2-1

- 3 copias de los datos
- 2 medios diferentes
- 1 copia fuera de la sede

4.2.2. Validación de respaldos

Cada semana debe realizarse:

- Importación de prueba del backup
- Verificación de integridad
- Revisión de logs de MySQL
- Simulación de restauración parcial

Esto garantiza que los backups no estén corruptos.

4.3. Restauración (Recover)

En caso de pérdida de datos:

1) Restaurar el último respaldo completo

```
mysql -u root -p sedam_reclamos < backup_sedam_full.sql
```

2) Aplicar respaldos incrementales

```
mysqlbinlog backup_incremental_1.sql | mysql -u root -p
```

3) Validar integridad

- Comparar número de registros

- Validar llaves foráneas
- Revisar coherencia del historial

4.4. Medidas de seguridad adicionales

1) Integridad referencial estricta

Ya implementada mediante:

- Foreign keys
- Cascadas específicas
- Restricciones ON DELETE / ON UPDATE

2) Transacciones ACID

MySQL InnoDB asegura:

- Atomicidad en registros críticos
- Prevención de datos huérfanos
- Manejo seguro de múltiples operadores

3) Prevención de SQL Injection

- Uso de consultas preparadas
- Validación de datos en backend
- Deshabilitar concatenación de SQL en controladores

4) Seguridad de evidencias

(Imagen, PDF, video)

- Almacenadas fuera del servidor público
- Validación de tamaño y tipo
- Antivirus de servidor
- Rutas protegidas

5) Monitoreo

- MySQL Slow Query Log
- MySQL Error Log
- Alertas en fallos de login
- Alertas en aumento de errores de FK

3. Capítulo 8. Diseño detallado de sistemas en red y móviles

3.1 Modelo de comunicación

3.1.1 Interacción entre la interfaz y el servidor

El sistema está construido bajo el modelo Cliente–Servidor, donde el navegador del usuario actúa como cliente y el servidor web procesa las solicitudes, gestiona la lógica del sistema y accede a la base de datos. En este modelo, el cliente se encarga únicamente de mostrar la información y enviar solicitudes, mientras que el servidor realiza todas las operaciones internas.

El sistema utiliza un mecanismo de comunicación basado en solicitudes HTTP/HTTPS y en el intercambio de datos en formato JSON, mediante el cual la aplicación web envía información al servidor cada vez que el usuario registra un reclamo, consulta su estado o actualiza información.

Este mecanismo permite que la interfaz web envíe datos, reciba respuestas y muestre información actualizada sin necesidad de instalar aplicaciones adicionales, pues todo se ejecuta desde el navegador y la lógica principal permanece centralizada en el servidor.

3.2. Justificación del modelo de comunicación

3.2.1. Justificación del modelo Cliente–Servidor

El sistema adopta el modelo Cliente–Servidor porque permite separar claramente la lógica de presentación que se ejecuta en el navegador del usuario y la lógica de negocio que reside en el servidor web. En este modelo, el cliente solo muestra la información y envía solicitudes, mientras que el servidor procesa los datos, gestiona los reclamos y se comunica directamente con la base de datos.

Este enfoque es el más adecuado para un sistema orientado a la gestión de reclamos ciudadanos, ya que centraliza toda la información en un único servidor, garantiza la integridad de los datos y facilita el mantenimiento, las actualizaciones y el control de seguridad. Además, permite que los usuarios accedan al sistema simplemente mediante un navegador, sin necesidad de instalar aplicaciones, lo cual es coherente con los objetivos del proyecto y con la accesibilidad que promueve el ODS 6.

El modelo Cliente–Servidor también mejora la eficiencia del sistema al reducir la carga en el cliente, evitando que el navegador tenga que procesar lógica compleja o conectarse directamente a la base de datos. Gracias a esta distribución de responsabilidades, el sistema opera de manera más estable, organizada y segura.

3.2.2 Justificación del protocolo

El sistema emplea los protocolos HTTP y HTTPS porque son los estándares más utilizados en aplicaciones web y permiten transmitir información entre la interfaz y el servidor de forma clara y estructurada. HTTP facilita operaciones como registrar reclamos, consultar estados, obtener historial y actualizar información mediante solicitudes GET, POST, PUT o DELETE.

El uso de HTTPS es fundamental porque cifra toda la información transmitida, protegiendo datos sensibles del ciudadano como nombres, direcciones y evidencias. Esto evita que los datos sean interceptados o manipulados durante su transmisión, garantizando un uso seguro incluso cuando el usuario se conecta desde redes públicas o móviles.

Gracias a estos protocolos, el sistema logra una comunicación confiable, segura y compatible con cualquier navegador o dispositivo.

3.2.3 Justificación del formato

El sistema utiliza JSON como formato de intercambio de datos porque es ligero, fácil de interpretar y ampliamente compatible con navegadores y aplicaciones web. Su estructura simple reduce el tiempo de carga y optimiza el consumo de datos, lo cual es especialmente útil cuando el ciudadano utiliza un dispositivo móvil o una conexión inestable. A diferencia de XML, JSON requiere menos etiquetas y, por lo tanto, ocupa menos espacio, lo que permite una comunicación más rápida. Además, puede ser procesado directamente por JavaScript sin necesidad de librerías adicionales, lo que agiliza el funcionamiento de la aplicación web. También es el formato estándar en APIs modernas y servicios web, lo que facilita su integración con futuras aplicaciones móviles o módulos adicionales. Por estas razones, JSON es el formato más eficiente y adecuado para asegurar que el sistema funcione de manera rápida, fluida y con un mejor rendimiento general.

3.3. Funcionamiento del modelo de comunicación

3.3.1. Ejemplo de la comunicación



Figura 1. En el ejemplo se observa de manera general como es que se comunican el usuario, interfaz web, servidor y base de datos.

3.3.2. Explicación de la comunicación

El diagrama ilustra el flujo general de comunicación del sistema. Primero, el usuario realiza una acción en la interfaz web, como registrar un reclamo o consultar su estado. La aplicación genera una solicitud HTTP o HTTPS en formato JSON y la envía al servidor. El servidor recibe esta solicitud, la procesa y accede a la base de datos para registrar o recuperar la información correspondiente. Una vez procesada la operación, el servidor envía una respuesta en formato JSON de regreso a la interfaz. Finalmente, la interfaz muestra la información actualizada al usuario, como confirmaciones, códigos de reclamo o estados del proceso.

3.4. Diagrama de Despliegue del Modelo Cliente–Servidor

El diagrama de despliegue muestra la estructura física del Sistema de Gestión de Reclamos, representando los dispositivos involucrados y la forma en que interactúan entre sí. En él se observa al cliente, que accede al sistema mediante un navegador web ejecutado en su dispositivo; el servidor web, donde se aloja la aplicación y la API REST encargada de procesar reclamos, gestionar notificaciones, realizar validaciones y manejar la comunicación mediante HTTP/HTTPS y datos en formato JSON; y finalmente, el servidor de base de datos, que almacena de forma estructurada toda la información del sistema mediante un motor SQL normalizado. Las conexiones representadas indican el flujo de comunicación real: el cliente se comunica con el servidor web mediante HTTPS en el puerto 443, y este a su vez interactúa con la base de datos a través de consultas SQL y transacciones ACID. El diagrama permite visualizar cómo se distribuyen los componentes del sistema en su arquitectura Cliente–Servidor, asegurando un funcionamiento seguro, organizado y coherente con la arquitectura en capas adoptada.

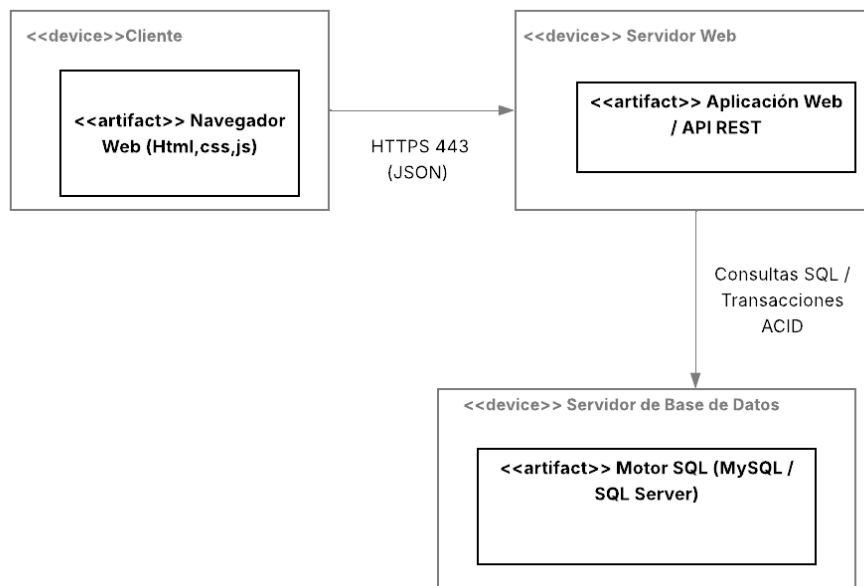


Figura 2. Diagrama de Despliegue UML: Interacción entre Cliente, Servidor Web y Servidor de Base de Datos

3.5 Diseño del sistema web o móvil

3.5.1. Estructura general del sistema web

El sistema está diseñado como una aplicación web accesible desde cualquier navegador, permitiendo que los ciudadanos, que se registraron o logearon con sus datos personales, registran reclamos y consulten su estado sin instalar software adicional. El diseño se basa en páginas web construidas con HTML, CSS y JavaScript, las cuales se comunican constantemente con el servidor mediante solicitudes HTTP/HTTPS y el uso de datos en formato JSON.

La estructura del sistema está organizada para mantener una navegación simple, directa y con una carga de información clara para el usuario, evitando elementos innecesarios que dificulten el uso del portal.

3.5.2 Diseño de la interfaz web

La interfaz del sistema web sigue criterios de: simplicidad, claridad visual, pocos pasos por acción, formularios ordenados, mensajes de retroalimentación visibles.

Los elementos se presentan de forma limpia, con textos explicativos y botones claramente identificados, tal como se haría en una aplicación móvil, pero adaptado al formato web. Este enfoque mejora la experiencia del usuario y permite que cualquier persona pueda usar el sistema sin dificultades.

3.5.3. Tolerancia a fallos y resiliencia en el sistema web

El sistema está preparado para seguir operando correctamente incluso cuando ocurren fallos temporales, como:

- Errores de conexión
- Lentitud del servidor
- Problemas momentáneos en la red del usuario

Cuando ocurre una falla, la interfaz web muestra mensajes claros indicando que la operación no se completó y permite reintentar el envío sin que el usuario pierda la información ingresada.

Del lado del servidor, las operaciones se hacen de forma controlada para evitar registros incompletos o inconsistentes cuando ocurre un fallo.

Gracias a estas medidas, el sistema puede recuperarse de errores puntuales y mantener la integridad de los datos y la continuidad del servicio.

3.5.4 Pruebas y validación del sistema web

Para asegurar el correcto funcionamiento del sistema, se realizarán actividades de pruebas y validación orientadas a verificar tanto la interacción del usuario como la comunicación con el servidor. Estas pruebas se aplicarán a los formularios, botones, mensajes y flujos principales del sistema.

Se validará que:

- Los formularios acepten solo datos válidos
- Las solicitudes HTTP/HTTPS se envíen correctamente
- Las respuestas JSON del servidor se reciban y muestren sin errores
- Las actualizaciones del estado del reclamo sean precisas
- El sistema reaccione de manera adecuada ante fallos de red o respuestas inválidas

El objetivo de estas pruebas es confirmar que el sistema web es estable, funcional y que cumple correctamente con los requerimientos de registro, consulta y actualización de reclamos.

3.6. Gestión de datos en red

La gestión de datos en red del Sistema de Gestión de Reclamos de Agua Potable se centra en asegurar el intercambio eficiente, seguro y consistente de la información entre la capa móvil, la capa web y el servidor central de SEDAM. Para ello se implementan las siguientes prácticas:

3.6.1. Transmisión de datos mediante API REST

Toda la información (usuarios, reclamos, estados, evidencias, notificaciones) se gestiona mediante servicios REST, los cuales permiten intercambiar datos en formato JSON de forma estructurada y controlada entre cliente y servidor.

3.6.2. Sincronización en tiempo real

Los reclamos, cambios de estado, evidencias y notificaciones se actualizan en tiempo real a través de peticiones HTTP seguras, evitando inconsistencias entre web y móvil.

El sistema garantiza que los datos mostrados al ciudadano y a los operadores estén siempre actualizados.

3.6.3. Sincronización en tiempo real

Para evitar conflictos cuando varios operadores gestionan un mismo reclamo, el servidor aplica:

- Validaciones de estado previo
- Transacciones ACID en la base de datos
- Bloqueos lógicos en operaciones críticas

Esto evita duplicidad de registros o actualizaciones incorrectas.

3.6.4. Sincronización en tiempo real

La base de datos se encuentra normalizada (3FN), lo que garantiza integridad referencial entre:

- usuario—ciudadano—operador
- reclamo—estado—historial
- reclamo—evidencias

- reclamo–notificaciones
- reportes–formatos oficiales

Esto permite que los datos viajen en red sin duplicación ni pérdida de integridad.

3.6.5. Sincronización en tiempo real

Antes de que los datos se envíen por la red:

- el cliente (web/móvil) valida formato y obligatoriedad
- el servidor valida reglas de negocio

Esto evita errores y reduce el tráfico innecesario en la red.

Seguridad en red y móviles

La seguridad en red y móviles se aplica para proteger la información transmitida, los dispositivos que se conectan al sistema y la infraestructura de SEDAM.

3.7.1 Cifrado de comunicación

Toda interacción entre aplicación móvil, web y servidor se realiza usando:

- HTTPS con TLS 1.2 o superior

Esto evita interceptaciones o manipulaciones de datos sensibles.

3.7.2 Seguridad de acceso para usuarios móviles

Los dispositivos móviles utilizan:

- almacenamiento seguro para el token de sesión
- expiración programada del token
- bloqueo de sesión ante intentos fallidos

Así se evita que terceros accedan si el dispositivo se pierde o es robado.

3.7.3 Protección contra ataques de red

Para evitar conflictos cuando varios operadores gestionan un mismo reclamo, el servidor aplica:

- Validaciones de estado previo
- Transacciones ACID en la base de datos
- Bloqueos lógicos en operaciones críticas

Esto evita duplicidad de registros o actualizaciones incorrectas.

3.7.4 Subida segura de evidencias

Las evidencias enviadas desde el móvil (fotos, PDF, videos):

- viajan cifradas
- son filtradas por tipo y tamaño
- pasan por un antivirus del servidor
- se almacenan en rutas seguras

Integridad de la aplicación móvil

Para evitar aplicaciones falsificadas o manipuladas:

- verificación de firma digital
- bloqueo de apps modificadas
- detección básica de root/jailbreak

3.7.6 Seguridad en servidor y red interna

El backend y la base de datos están aislados en una red interna privada, evitando accesos directos desde internet y reduciendo riesgos de intrusión.

3.8 Justificación técnica

La implementación del Sistema de Gestión de Reclamos de Agua Potable se basa en un conjunto de decisiones técnicas que garantizan la eficiencia, seguridad, compatibilidad y escalabilidad del sistema. Cada elemento del diseño —desde la comunicación entre la interfaz y el servidor hasta la gestión de datos en red y la seguridad en entornos móviles— ha sido seleccionado para asegurar un funcionamiento confiable y adaptado al contexto institucional de SEDAM.

3.8.1 Justificación de la arquitectura técnica

El uso de una arquitectura en capas (N-Tier) es apropiado debido a que separa claramente la presentación, la lógica de negocio y el acceso a datos, lo cual facilita el mantenimiento y reduce el riesgo de errores entre componentes. Esta estructura permite que la interfaz web funcione de manera independiente del motor de base de datos y que la lógica de negocio pueda modificarse sin afectar la visualización o la infraestructura.

Además, esta arquitectura se ajusta al tamaño y a la naturaleza centralizada de SEDAM, evitando la complejidad innecesaria de arquitecturas distribuidas como microservicios.

3.8.2 Justificación del modelo de comunicación

La elección de HTTP/HTTPS como protocolo principal responde a su compatibilidad universal con cualquier navegador o dispositivo, permitiendo que el sistema funcione sin instalaciones adicionales. El uso de HTTPS garantiza comunicaciones cifradas, especialmente relevantes cuando los ciudadanos transmiten datos sensibles o cargan evidencias desde redes móviles o públicas.

El formato JSON complementa esta elección, ya que es ligero, eficiente y procesado nativamente por JavaScript, optimizando el rendimiento y reduciendo el consumo de datos de los usuarios móviles.

3.8.3 Justificación del diseño web y su estructura

El enfoque de diseño simple y orientado a la claridad facilita el acceso de todo tipo de usuarios, incluyendo ciudadanos con poca experiencia

tecnológica. Formularios ordenados, retroalimentación inmediata y navegación directa reducen las posibilidades de error y hacen que el registro de reclamos y la consulta de estados sean procesos rápidos y comprensibles.

La estructura modular del sistema web se alinea con las buenas prácticas de diseño responsivo, asegurando funcionamiento en computadoras y dispositivos móviles sin desarrollar aplicaciones nativas adicionales.

3.8.4 Justificación de la tolerancia a fallos.

El sistema incorpora mecanismos para mantener su operatividad incluso frente a condiciones adversas, como caídas temporales del servidor o baja estabilidad de red.

Medidas como reintentos controlados, mensajes claros al usuario, transacciones ACID y bloqueos lógicos evitan pérdidas de información o inconsistencias en los reclamos.

Esto es crucial porque el sistema maneja datos oficiales y evidencias, cuyos registros deben ser precisos y verificables.

3.8.5 Justificación de la gestión de datos en red

La decisión de utilizar servicios REST permite un intercambio de datos estructurado, seguro y estándar entre la interfaz web, el servidor y la base de datos.

El control de concurrencia impide que varios operadores generen inconsistencias al gestionar un mismo reclamo.

Además, la normalización de la base de datos y su uso en red aseguran integridad referencial y un manejo eficiente de la información institucional, facilitando auditorías, reportes y trazabilidad completa del proceso.

3.8.6 Justificación de las medidas de seguridad en red y móviles

La seguridad se aborda desde varios niveles:

- cifrado TLS para proteger toda la comunicación,
- tokens de sesión seguros en móviles,
- controles de roles para restringir funciones entre ciudadanos y operadores,
- firewalls y validación interna para evitar ataques externos,
- verificación de evidencias para prevenir archivos maliciosos.

Estas medidas son indispensables porque el sistema maneja datos personales, reportes institucionales y material multimedia que debe protegerse contra accesos no autorizados y modificaciones indebidas. La protección de dispositivos móviles y la integridad de la aplicación fortalecen el uso seguro en campo por parte de operadores y ciudadanos.

3.8.7 Justificación del uso de patrones de diseño

La elección de patrones como DAO, Observer y Fachada permite mantener un sistema modular, fácil de expandir y técnicamente sólido:

- DAO separa la lógica de negocio del acceso a datos.
- Observer automatiza notificaciones sin acoplar módulos.
- Fachada simplifica la coordinación de procesos complejos como el registro de reclamos.

Estos patrones reducen errores, facilitan la reutilización del código y garantizan un funcionamiento claro y estandarizado en todos los módulos.

4. Referencias

- [1] PRESSMAN, Roger S. y MAXIM, Bruce R. *Ingeniería del software: Un enfoque práctico*. 8.^a ed. Nueva York: McGraw-Hill, 2014. Disponible en: <https://www.mheducation.com/highered/product/software-engineering-practitioner-s-approach-pressman-maxim/M9780078022128.html>. [Consulta: 16 nov. 2025].
- [2] GAMMA, Erich; HELM, Richard; JOHNSON, Ralph y VLISSIDES, John. *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1994. Disponible en: <https://learning.oreilly.com/library/view/design-patterns-elements/0201633612/>. [Consulta: 16 nov. 2025].
- [3] FOWLER, Martin. *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley, 2002. Disponible en: <https://martinfowler.com/books/ea.html>. [Consulta: 16 nov. 2025].
- [4] BOOCH, Grady; RUMBAUGH, James y JACOBSON, Ivar. *The Unified Modeling Language User Guide*. 2.^a ed. Boston: Addison-Wesley, 2005. Disponible en: <https://learning.oreilly.com/library/view/the-unified-modeling/0201571684/>. [Consulta: 16 nov. 2025].

[5] SILBERSCHATZ, Abraham; KORTH, Henry F. y SUDARSHAN, S. *Database System Concepts*. 7.^a ed. Nueva York: McGraw-Hill, 2020. Disponible en: <https://www.db-book.com/>. [Consulta: 16 nov. 2025].

[6] OWASP FOUNDATION. *OWASP Top Ten 2021 [en línea]*. 2021. Disponible en: <https://owasp.org/www-project-top-ten/> [Consulta: 16 nov. 2025].